

Python Grundlagen

Bisher haben wir lediglich mit den Befehlen aus dem turtle-Modul gearbeitet, um eine kleine Schildkröte zu steuern und Figuren zu generieren. Python kann jedoch viel mehr als nur dieses grafische Modul.

Wie jede Programmiersprache folgt auch Python bestimmten Regeln und Muster, damit wir sinnvoll mit dem Computer arbeiten können. Diese Elemente werden wir uns genauer anschauen.

Grundelemente einer Programmiersprache

1. Variablen und Zuweisungen

Der Computer muss sich Sachen merken können.

2. Ein- und Ausgabe

Der Computer muss Input entgegennehmen können und Output ausgeben können.

3. Operatoren

Der Computer muss wissen, was er mit den verschiedenen Elementen machen kann (rechnen, vergleichen, ...).

4. Datentypen

Der Computer hat Regeln, was er mit welchen Elementen machen darf.

5. Kontrollstrukturen

Der Computer muss Entscheidungen treffen und Dinge wiederholen können.

6. Funktionen

Der Computer muss Aufgaben bündeln und wiederverwenden können.

All diese Elemente sind eng miteinander verbunden und manchmal ist es schwierig das eine zu verstehen, ohne die anderen. Darum versuchen wir in kleinen Etappen sie alle näher zu begreifen.

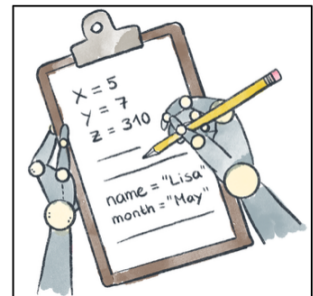
1. Variablen und Zuweisungen

Damit der Computer ein Programm ausführen kann, muss er sich Sachen merken können. Wie wenn wir Menschen beim Einkaufen eine Einkaufsliste brauchen. Die gespeicherten Werte nennt man **Variablen**.

Variablen und Zuweisungen

Eine **Variable** benutzt man, um einen Wert zu speichern (wie ein Platzhalter).

Sie wird mit einer **Zuweisung** (=) erstellt, wobei links der Name und rechts der Wert der Variable steht.



Zuweisungen



```
1 x = 5
2 monat = "Januar"
3 summe = 1 + 256 + 42
```

Die **Werte der Variablen** können Zahlen, Text oder auch Berechnungen sein.

In der ersten Zeile erhält die Variable x den Wert 5. In der zweiten Zeile bekommt die Variable monat den Wert "Januar". Dies ist ein Text, der immer in Anführungszeichen stehen muss, damit der Computer ihn als solchen versteht.

Den **Namen der Variablen** darf man (fast) frei wählen:

- erlaubt sind Buchstaben (klein und gross), Zahlen und der Unterstrich _
- der Name darf nicht mit einer Zahl beginnen

Im Verlauf des Programms können sich die Werte von Variablen ändern (man kann sie also überschreiben).

Variablen überschreiben



```
1 x = 5
2 x = 7 # der Wert von x ist neu 7
```

Mit dem Symbol # können wir Code in Python **kommentieren**. Alles, was nach dem # auf derselben Zeile folgt, wird vom Computer ignoriert. **Kommentare** helfen uns, den Code lesbarer (verständlicher) zu machen.

Aufgabe 1

Gegeben ist eine Liste von möglichen **Variablen**namen. Welche sind laut den Regeln für Python gültig?

Summe_1 = 433	summe_1 = 123	summe1 = 63
*myname = "Glurak"	_my_name = "Pikachu"	my name = "Flegmon"
8Bit = 256	L337Code = 42	Byte8Numb3r = 16

Wie kann man schnell überprüfen, ob ein Variablenname okay ist oder nicht?

2. Ein- und Ausgabe

Damit wir mit dem Computer arbeiten können, braucht eine Programmiersprache zwei Dinge: Wir müssen **Eingaben (Input)** machen können, und der Computer muss diese verarbeiten und uns danach **Resultate (Output)** zurückgeben.

In Python gibt es zwei Funktionen (Befehle) dafür.

`input()` Damit fragt der Computer nach einem Input vom Benutzer.

`print()` Damit können Sie beliebige Sachen in die Kommandozeile (Ausgabefenster) ausdrucken lassen.



Benutzer-Eingabe und Resultat-Ausgabe



```
1 erste_zahl = input("Erste Zahl:")
2 zweite_zahl = input("Zweite Zahl:")
3 print(erste_zahl + zweite_zahl)
```

Um Programme wirklich nützlich machen zu können, braucht es meistens **Inputs** vom Benutzer und ein Resultat als **Output**, das für den Benutzer hilfreich ist.

Beispiele print() und input()

print()

Testen Sie mal folgende Zeilen, indem Sie sie in Ihrem Thonny laufen lassen. Stimmt das Ergebnis mit Ihren Erwartungen überein?

```
1 print("2+3")
```

```
1 print(2+3)
```

Wenn Sie Text und Zahlen gleichzeitig drucken möchten, müssen Sie es mit einem Komma trennen:

```
1 print("Die Summe ist: ", 5)
```

input()

Testen Sie auch den neuen Befehl `input()`. Was passiert bei der folgenden Zeile?

```
1 input("Wie heisst du? ")
```

Wenn man möchte, dass der Benutzer als Input nur eine ganze Zahl eingeben darf, müssen wir noch folgende Anpassung vornehmen:

```
1 int(input("Wie alt bist du?"))
```

Falls der Benutzer keine Zahl eingibt, kommt ein Fehler.

Jetzt können wir auch alles miteinander verbinden, was wir bis jetzt gelernt haben:

```
1 name = input("Wie heisst du? ")
2 print("hallo"*2,name)
```

Aufgabe 2

Erstellen Sie eine Variable mit dem Namen `my_age`. Weisen Sie ihr den Wert Ihres Alters zu. Damit Sie sehen, ob es funktioniert hat, benutzen Sie in der nächsten Zeile `print(my_age)`, um die Variable auszugeben.

Aufgabe 3

Können Sie das Programm so anpassen, dass der Computer Sie mit `input()` fragt, wie alt Sie sind und er dann die Zahl wieder ausgibt? Die Kommandozeile sollte so aussehen:

```
>>> %Run -c $EDITOR_CONTENT
Wie alt sind Sie?29
Ihr Alter ist: 29
```

Lösungen

Lösung 1

gültig:

```
Summe_1 = 433
summe_1 = 123
_my_name = "Pikachu"
L337Code = 42
summe1 = 63
Byte8Numb3r = 16
```

ungültig:

```
*myname = "Glurak"
8Bit = 256
my name = "Flegmon"
```

Variablenname auf Gültigkeit überprüfen:

Zuweisung mit dem Variablennamen in Thonny eingeben und den Code laufen lassen. Bei einem ungültigen Namen wird eine Fehlermeldung erscheinen.

Lösung 2

```
1 my_age = 17
2 print(my_age)
```



Lösung 3

```
1 my_age = input("Wie alt sind Sie?")
2 print("Ihr Alter ist", my_age)
```

